

ChromGraphFM

Explained from zero

*Can a DNA model learn not only the letters of DNA,
but also which distant regions physically touch inside a cell?*

This document is written for two readers at once.

If you have **never seen a DNA sequence or a neural network**, start at page one and read straight through. Nothing is assumed. Every term is defined the first time it appears, and every equation is preceded by the same idea in ordinary English.

If you are a **researcher**, Parts III and IV are the substance: the mechanism, its derivation, and a complete numerical trace of one sample through the model. Part V states what would falsify the claim.

B.Tech Mini Project • 23CS7552
Department of Computer Science and Engineering
VR Siddhartha Engineering College, Vijayawada

P. Charan Sai • G. Karthik • P. Sanath
Under the guidance of Mrs. K. Divya Kothapali

Contents

How to read this document	2
Part I — The biology	2
1 What DNA is	2
2 Why folding matters	3
3 Hi-C: measuring the folding	4
Part II — The machine learning	5
4 Turning things into vectors	5
5 Attention, from scratch	6
6 Self-supervised pretraining	7
7 Graphs	7
Part III — The model	8
8 The idea in one picture	8
9 Step 1 — windows and nodes	8
10 Step 2 — the local encoder	9
11 Step 3 — building the sparse contact graph	9
12 Step 4 — contact-biased attention	10
13 Step 5 — structure dropout	11
14 Step 6 — the three training objectives	12
Part IV — Worked example	13
15 The setting	13
16 Stage 1 — DNA becomes vectors	14
17 Stage 2 — Hi-C becomes contact strengths	14

18 Stage 3 — the graph	15
19 Stage 4 — contact-biased attention, in full	15
20 Stage 5 — residual connection and normalisation	16
21 Stage 6 — feed-forward, and the final representation	17
22 Stage 7 — the three losses	17
23 Stage 8 — how the model learns to trust Hi-C	18
Part V — Evaluation	19
24 The controls	19
25 Hi-C-free inference — and a trap in the comparison	20
26 What would falsify the claim	20
27 Honest positioning	20
Appendix	21
A Notation	21
B Glossary	21
C Reproducing the worked example	23

How to read this document

Five kinds of box appear throughout. You can read the whole document using only the green boxes and the ordinary text; the blue boxes add the mathematics.

In plain words

The idea in ordinary English, with no symbols. If you read nothing else, read these.

The maths

The same idea written as an equation, with every symbol named. Skip these on a first pass if you like — nothing later depends on your having read them.

Why it is built this way

Why the model is built this way rather than some simpler way. These are the design decisions, and they are where the research actually lives.

Easy to get wrong

A mistake that is easy to make and hard to notice. Several of these describe errors that were found and fixed while preparing this document.

Check it yourself

A small calculation you can do on paper to convince yourself the previous section is true.

Part I — The biology, starting from nothing

1 What DNA is

DNA is a long molecule that carries the instructions for building and running a living thing. For our purposes it can be treated as a very long piece of text written in an alphabet of exactly four letters:

A C G T

These letters are called **bases** or **nucleotides**. A stretch of DNA looks like this:

... A T G C A T G C C C T C A G T T T A T A T A A ...

The human genome — the complete DNA of one human — is about **3.1 billion** letters long. Written out at normal book size it would fill roughly a million pages.

A **genome assembly** is an agreed reference copy of that text. Ours is called hg38: *human genome, version 38*. When this project says “position 1,250,000 on chromosome 3”, it means a specific letter in that agreed reference.

1.1 Genes, promoters and enhancers

Not all of the text does the same job.

- A **gene** is a stretch that codes for a product — usually a protein.
- A **promoter** sits immediately in front of a gene. It is the switch: the cell's machinery attaches here to start reading the gene.
- An **enhancer** is a separate stretch that makes a promoter more active. It is a volume dial rather than an on/off switch.

Here is the fact that this entire project exists because of:

In plain words

An enhancer usually is not next to the gene it controls. It can sit a million letters away in the text — with dozens of other genes in between — and still control that one specific gene and not its neighbours.

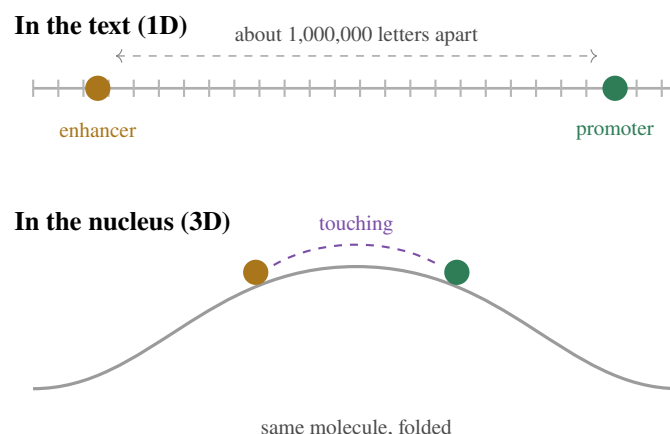
If you only read the text left to right, you have no way of knowing which enhancer belongs to which gene.

2 Why folding matters

The human genome is about two metres of DNA if you stretched it out. It has to fit inside a cell nucleus about six micrometres across — roughly the ratio of fitting two kilometres of thread into a tennis ball.

So it folds. And crucially, **it does not fold randomly**. The folding is organised, it is repeatable between cells of the same type, and it is different between cell types.

When DNA folds, two stretches that are far apart in the text can end up touching in space:



That contact is how a distant enhancer reaches its gene. Physical closeness, not textual closeness, is what decides which enhancer controls which promoter.

In plain words

Two different meanings of “near”.

Near in sequence means few letters apart in the text.

Near in space means physically touching inside the cell.

They are not the same thing, and it is the second one that controls genes. A model that only reads the text sees the first one only.

3 Hi-C: measuring the folding

We do not have to guess how the genome folds. There is an experiment that measures it, called **Hi-C**.

The idea, simplified to its essentials:

1. Freeze the DNA in place so that stretches which are touching stay stuck together.
2. Cut the DNA into fragments.
3. Glue together the fragment ends that were touching.
4. Read the glued pairs. Each pair says “these two places in the genome were close together”.
5. Count how often each pair of places shows up.

The result is a **contact map**: a big table of counts.

- The genome is chopped into equal-sized **bins** — ours are **5 kb** each, meaning 5,000 letters.
- Row i and column j of the table hold the number of times bin i and bin j were found touching.
- A big number means those two regions are often close in space.

In plain words

Two maps of the same territory.

The *DNA sequence* answers: *what is written in the genome?*

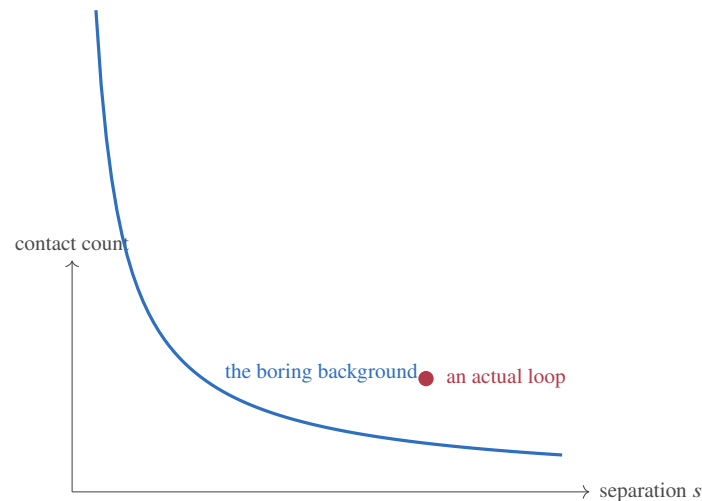
The *Hi-C contact map* answers: *which parts of the genome are physically close?*

Most DNA models only ever read the first map. This project gives the model both, and — this is the part that matters — lets the second one change how it reads the first.

3.1 The single most important property of a contact map

Contact counts are dominated by one boring fact: **regions that are close in the text touch more often, simply because they are attached.** Two bins right next to each other will have a huge count. Two bins a megabase apart will have a small one, whether or not anything interesting connects them.

If you plot count against separation you get a steep downward curve. It is usually written $P(s)$ — the probability of contact at separation s .



The interesting biology — a real enhancer–promoter loop — is the little bump sitting *above* that curve. To find it you have to divide the background out. That step is called **distance detrending**, and Section 11 does it with actual numbers.

Easy to get wrong

If you skip detrending, then “the strongest contacts” just means “the nearest bins”, every time. A model fed those would learn the rule *things close in the text touch*, which it could already work out from the input, and you would have measured nothing.

This is the single most common way to accidentally produce a meaningless result in this field.

Part II — The machine learning, starting from nothing

4 Turning things into vectors

A neural network cannot work on the letter A. It works on numbers. So the first job is always to turn the thing you care about into a list of numbers.

A **vector** is just an ordered list of numbers, like [0.42, 0.15, 0.12, 0.07]. The number of entries is its **dimension**.

The simplest way to turn a base into a vector is **one-hot encoding**: use four slots, one per letter, and put a 1 in the slot for the letter you have.

	A	C	G	T
A	1	0	0	0
C	0	1	0	0
G	0	0	1	0
T	0	0	0	1

An **embedding** is a vector that a model has *learned*, rather than one you wrote down by hand. After

training, similar things end up with similar embeddings. That is the whole point: the vector becomes a compressed summary of what the model has worked out about that thing.

In plain words

An embedding is the model’s private summary of something, written as a list of numbers. Two DNA regions that behave similarly should end up with similar summaries, even if their letters differ.

5 Attention, from scratch

Attention is the mechanism that lets one part of the input look at other parts and decide how much to take from each. Almost every modern language model is built on it, and our model modifies it, so it is worth understanding properly.

Suppose we have three regions, each with an embedding, and region i wants to gather information from the others. Attention does it in four steps.

Step 1 — ask a question, publish an advertisement

From each region’s embedding we compute three new vectors using three learned matrices:

- the **query** $\mathbf{q}_i$ — “here is what I am looking for”;
- the **key** $\mathbf{k}_j$ — “here is what I have to offer”;
- the **value** $\mathbf{v}_j$ — “here is the actual content I would hand over”.

Step 2 — score every candidate

The score of region j from region i ’s point of view is the **dot product** $\mathbf{q}_i^\top \mathbf{k}_j$: multiply the two vectors entry by entry, then add up. A large score means the question and the advertisement match well.

Step 3 — turn scores into weights that add to one

Raw scores can be any size. The **softmax** function converts a list of scores into a list of positive weights summing to 1, so they can be read as proportions: exponentiate each score, then divide each by the total.

Step 4 — take a weighted average

The output for region i is the sum of every value vector, each multiplied by its weight. Regions with high weight contribute most.

The maths

For a query at region i over a set of candidate regions $\mathcal{N}(i)$:

$$a_{ij} = \frac{\exp(s_{ij})}{\sum_{m \in \mathcal{N}(i)} \exp(s_{im})}, \quad s_{ij} = \frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d}}, \quad \text{output}_i = \sum_{j \in \mathcal{N}(i)} a_{ij} \mathbf{v}_j. \quad (1)$$

Here d is the dimension of the vectors. Dividing by $\sqrt{d}$ keeps the scores from growing large just because the vectors are long, which would make the softmax output almost all-or-nothing and stop

the model learning.

Easy to get wrong

Softmax ignores anything added equally to every candidate.

Suppose you add the same constant κ to every score s_{ij} for a fixed i . Then

$$\frac{e^{s_{ij}+\kappa}}{\sum_m e^{s_{im}+\kappa}} = \frac{e^\kappa e^{s_{ij}}}{e^\kappa \sum_m e^{s_{im}}} = \frac{e^{s_{ij}}}{\sum_m e^{s_{im}}}.$$

The constant cancels exactly. **It has no effect whatsoever.**

Remember this. In Section 12 it stops being a curiosity and becomes a bug we had to find — and it is why one term that appeared in an earlier draft of this architecture was removed.

6 Self-supervised pretraining

To train a model you normally need labelled examples: an input and the right answer. Labels are expensive. In genomics they are very expensive, because each one is an experiment.

Self-supervised learning avoids this by hiding part of the input and asking the model to reconstruct it. The data labels itself.

In plain words

Cover up part of what you already have, ask the model to guess it, and score the guess. No human ever writes down an answer, but the model still gets a right and wrong on every example.

Pretraining means doing this on a large amount of data first, to learn general structure. **Fine-tuning** means then adapting the pretrained model to a specific task with far fewer labels. A **foundation model** is a model built to be pretrained once and reused for many tasks.

Easy to get wrong

Our model has roughly 9 million adjustable numbers. Genuine industrial foundation models have thousands of times more. We call ours a **compact pretrained representation model**, and only call it a foundation-model prototype if it demonstrably transfers to several tasks. Claiming otherwise in a report is an easy way to lose a reviewer's trust on page one.

7 Graphs

A **graph** is a set of things (**nodes**) and the connections between them (**edges**). A road map is a graph: cities are nodes, roads are edges.

Edges can carry information of their own. A road has a length and a speed limit; in our graph an edge will carry how strongly two genomic regions touch and how far apart they are in the text.

A graph is **sparse** when most possible connections are absent. With 6 nodes there are 15 possible edges; with 128 nodes there are 8,128. Keeping all of them is expensive and mostly pointless, because most pairs of genomic regions never touch. Ours keeps a small chosen subset.

Part III — The model

8 The idea in one picture

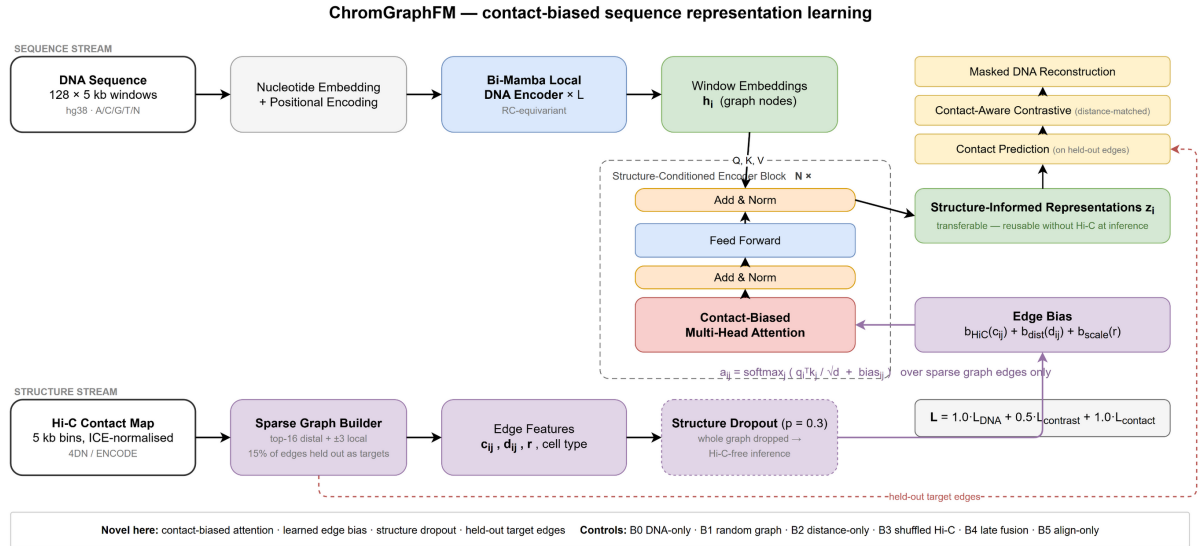


Figure 1: ChromGraphFM. The upper lane reads DNA; the lower lane turns Hi-C into a sparse graph. The two meet inside the encoder block, where contact strength is added to the attention score. Read it as two streams that meet in the middle.

The sequence stream

cuts the genome into windows, reads each one, and produces one vector per window.

The structure stream

turns the Hi-C map into a sparse graph over those same windows.

The meeting point

is attention: the graph decides which windows may exchange information, and the contact strengths decide how much.

Why it is built this way

There are three places you could put Hi-C, and only one of them answers our question.

After the model (*late fusion*): encode DNA, encode Hi-C, glue the two summaries together at the end. The DNA representation was already finished before Hi-C arrived, so Hi-C cannot have shaped it.

As the answer (*prediction target*): train the model to output the Hi-C map. Useful, but it produces a Hi-C predictor, not a reusable representation of DNA.

Inside the model (*our approach*): the contact map changes how the DNA windows talk to each other, at every layer, while the representation is still being formed.

Only the third one can answer “does knowing the folding change how the model reads the sequence?”

9 Step 1 — windows and nodes

We cut the region into equal windows of 5 kb — exactly the same size as the Hi-C bins.

Why it is built this way

Making the DNA window and the Hi-C bin the same size is not a detail. If they differed we would need a pooling layer to convert between them, and every coordinate calculation would acquire an opportunity to be off by one. Genomics code fails on off-by-one coordinate errors more than on anything else, and such a failure is silent: the model still trains, the loss still falls, and the result is simply wrong.

One window = one bin = one graph node. No conversion, no class of bug.

The real model uses 128 windows at once, covering 640 kb. The worked example in Part IV uses 6 windows of 8 letters so that every number fits on the page.

10 Step 2 — the local encoder

Each window's DNA goes through an encoder that outputs one vector for the whole window.

In plain words

The encoder reads 5,000 letters and writes a short summary of them — in the real model, 256 numbers. It is looking for things like transcription factor binding motifs, promoter-like patterns, and enhancer-like patterns.

At this stage each window is read *on its own*. No window can see any other window yet. All communication is saved for the graph.

We use a **bidirectional Mamba** encoder. Mamba is a **state space model**: it sweeps along the sequence carrying a running memory, updating it at each letter.

The maths

A state space model keeps a hidden state $\mathbf{s}_t$ and updates it letter by letter:

$$\mathbf{s}_t = \mathbf{A}_t \mathbf{s}_{t-1} + \mathbf{B}_t x_t, \quad y_t = \mathbf{C}_t^\top \mathbf{s}_t.$$

x_t is the letter at position t , $\mathbf{s}_t$ is what the model remembers after seeing it, and y_t is what it outputs there. In Mamba the matrices $\mathbf{A}_t, \mathbf{B}_t, \mathbf{C}_t$ depend on the input, which is what lets the model choose what to remember and what to forget.

The window vector is obtained by averaging the outputs across the window: $\mathbf{h}_i = \frac{1}{L} \sum_{t=1}^L y_t$.

Why it is built this way

Why not attention inside the window too? Attention costs time proportional to L^2 . At $L = 5,000$ letters that is 25 million pairs per window, per layer. A state space model costs time proportional to L . That is the difference between a model that fits on our GPU and one that does not.

Why bidirectional? DNA has no reading direction — it is double stranded, and a motif is a motif read either way. A one-directional scan would throw away half the context for no reason.

11 Step 3 — building the sparse contact graph

Now the Hi-C map becomes a graph. Four sub-steps.

3a — detrend against genomic separation

For each separation s , compute the average count seen at that separation. Then divide each observed count by the average for its own separation. The result is called **observed over expected**, or O/E. A value of 1.0 means “exactly as much contact as is normal at this distance”. A value of 2.0 means twice the normal amount — something real is going on.

3b — squash into a bounded range

O/E can be any positive number. We map it into $(0, 1)$ with $c = O/E / (1 + O/E)$, so the bias function later receives a well-behaved input.

3c — keep only a few edges per node

Each node keeps: all **local** neighbours within a small radius along the sequence, plus its **top- k** strongest *distal* partners, where distal means separated by at least a minimum number of bins.

3d — hold some edges back as answers

A fraction of edges is removed from the graph entirely and set aside. The model never sees them. They become the targets for the contact-prediction objective.

Easy to get wrong

Step 3d is not optional bookkeeping. If an edge is both an input to the model and the thing the model must predict, the model can achieve a perfect score by copying, and the number you report measures nothing at all.

The held-out edges are chosen with a *fixed* random seed, separate from the seed used for training. That way every training run sees exactly the same hold-out, and differences between runs cannot be confused with differences between hold-outs.

12 Step 4 — contact-biased attention

This is the mechanism the project exists to test.

In plain words

Ordinary attention decides how much window i should listen to window j using the DNA alone. We add two more terms before the decision is made:

- **how strongly they actually touch** — pushes the weight up;
- **how far apart they are in the text** — pushes the weight down.

Then attention proceeds as normal. The contact map does not get added to the answer at the end; it changes who is allowed to speak to whom, in every layer, while the answer is still being formed.

The maths

$$a_{ij} = \text{softmax}_{j \in \mathcal{N}(i)} \left(\underbrace{\frac{\mathbf{q}_i^\top \mathbf{k}_j}{\sqrt{d}}}_{\text{from the DNA}} + \underbrace{b_{\text{HiC}}(c_{ij})}_{\text{from the contacts}} + \underbrace{b_{\text{dist}}(d_{ij})}_{\text{from the separation}} \right) \quad (2)$$

with

- c_{ij} — the detrended, squashed contact strength between windows i and j ;
- $d_{ij} = |i - j|$ — their separation in bins;
- b_{HiC} and b_{dist} — small *learned* functions;
- $\mathcal{N}(i)$ — the graph neighbours of i , and nothing else.

Why it is built this way

Why is b_{dist} there at all, when we already detrended?

Detrending removed distance from the *edge selection*. But the model still needs to be able to treat a neighbour 3 bins away differently from one 40 bins away, because the biology differs. Giving it an explicit, separate distance term means the two effects can be told apart — and it is what makes the distance-only control experiment (Section 24) constructible: keep b_{dist} , delete b_{HiC} , and see what is lost.

Why learned functions rather than fixed ones? Because how much a given contact strength should matter is precisely the thing nobody knows. Fixing it would be assuming the answer.

Easy to get wrong

A term that did nothing. An earlier version of this architecture had a third bias term, $b_{\text{scale}}(r)$, where r was the resolution of the contact map.

We use a single resolution, so r is the same for every edge, so $b_{\text{scale}}(r)$ added the same constant to every candidate — and by Box 5 on page 7, softmax cancels it exactly. The term was mathematically incapable of affecting anything, while sitting in the headline equation of the project.

It has been removed. It becomes meaningful only if a second resolution is added, because then r varies between the edges of one node.

13 Step 5 — structure dropout

During training, with probability p , we throw the entire distal graph away for that sample and let the model work from local edges alone.

Why it is built this way

The central claim of the project is that the model *internalises* the structure — that after pretraining, the DNA encoder is better even when no Hi-C is available.

To test that, we must run the model without the graph. But if it had never once experienced life without the graph, removing it at test time would be an input unlike anything it saw in training, and a poor score would tell us nothing about what it learned.

Structure dropout makes the no-Hi-C setting a normal, familiar condition.

Easy to get wrong

Structure dropout **encourages** robustness. It does not **guarantee** that structural knowledge ended up stored in the sequence representation. Whether that happened is an experimental question, and Section 25 is the experiment. Do not write the guarantee into the report.

14 Step 6 — the three training objectives

The model is trained on three jobs at once.

14.1 Objective 1 — rebuild a hidden window

We blank out one window’s summary entirely and ask the model to reconstruct it from its graph neighbours.

The maths

$$\mathcal{L}_{\text{DNA}} = \frac{1}{d} \|\hat{\mathbf{h}}_m - \mathbf{h}_m\|^2, \quad \hat{\mathbf{h}}_m = \text{reconstruct}(\{\mathbf{h}_j : j \in \mathcal{N}(m)\}) \quad (3)$$

where m is the masked window and $\|\cdot\|^2$ is the sum of squared differences.

Easy to get wrong

This replaces per-letter masking, and the reason is arithmetic.

The original plan masked individual bases and reconstructed them from the window summary $\mathbf{h}_i$. But $\mathbf{h}_i$ is 256 numbers, and a window is 5,000 letters — 10,000 bits of information. You cannot recover 10,000 bits from 256 numbers. The objective was not weak, it was *impossible*, and its loss would have sat flat forever while looking like a training problem.

Masking the whole window and rebuilding it from neighbours is well posed, and it tests the mechanism more directly: the only route to the answer runs through the contacts.

14.2 Objective 2 — contacting regions should look alike

Windows that genuinely touch should get similar representations; windows that do not should get different ones.

The maths

$$\mathcal{L}_{\text{contrast}} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau) + \sum_{k \in \mathcal{K}} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)} \quad (4)$$

(i, j) is a touching pair, $\mathcal{K}$ is a set of non-touching ones, sim is cosine similarity, and τ is a temperature controlling how sharply the model must separate them.

Easy to get wrong

The negatives must be at the same genomic separation as the positive.

If you sample them freely, the non-touching pairs will on average be further apart than the touching pair. The model can then get a perfect score by measuring distance — something it can already read straight off the input — and you will have learned nothing about 3D structure.

Distance matching is not a refinement here. It is the difference between an experiment and a non-experiment.

14.3 Objective 3 — predict the contacts we hid

For each held-out edge, predict its strength from the two window representations and their separation.

The maths

$$\hat{c}_{ij} = \sigma(g(\mathbf{z}_i, \mathbf{z}_j, d_{ij})), \quad \mathcal{L}_{\text{contact}} = -\left[c_{ij} \log \hat{c}_{ij} + (1 - c_{ij}) \log(1 - \hat{c}_{ij})\right] \quad (5)$$

g is a small network, and $\sigma(x) = 1/(1 + e^{-x})$ squashes its output into $(0, 1)$ so it can be compared with c_{ij} .

14.4 Putting them together

The maths

$$\mathcal{L} = \lambda_{\text{DNA}} \mathcal{L}_{\text{DNA}} + \lambda_{\text{contrast}} \mathcal{L}_{\text{contrast}} + \lambda_{\text{contact}} \mathcal{L}_{\text{contact}} \quad (6)$$

with $\lambda_{\text{DNA}} = 1.0$, $\lambda_{\text{contrast}} = 0.5$, $\lambda_{\text{contact}} = 1.0$, tuned on validation chromosomes only.

Part IV — One sample, all the way through

15 The setting

Everything in this part is real arithmetic on a deliberately tiny model, so that you can check any step with a calculator. Every number is produced by `docs/worked_example.py` and inserted here automatically — none was typed by hand.

	This example	The real model
Windows per sample	6	128
Letters per window	8	5,000
Vector dimension d	4	256
Attention heads	1	8
Local radius	1	3
Distal edges per node	1	16

15.1 The DNA

Window	Sequence	Note
0	ATGCATGC	
1	CCCTCAGT	carries a CTCF-like motif core, CCCTC
2	TTATATAA	the window we will follow
3	GCGCGCGC	
4	AATTAATT	
5	TGAGGGCA	carries the reverse complement, GAGGG

Windows 1 and 5 are the interesting pair. In real chromatin, a CTCF motif and its reverse complement facing each other is the classic signature of a loop anchor. We have built that into the toy so there is something true for the model to find.

16 Stage 1 — DNA becomes vectors

Window 2 is TTATATAA. One-hot encoded, then reduced to base composition (what fraction of the window is each letter):

$$\mathbf{f}_2 = \begin{bmatrix} 0.500 \\ 0.000 \\ 0.000 \\ 0.500 \end{bmatrix} \begin{array}{l} \leftarrow \text{A} \\ \leftarrow \text{C} \\ \leftarrow \text{G} \\ \leftarrow \text{T} \end{array}$$

Passing this through the encoder gives the window vector:

$$\mathbf{h}_2 = \tanh(\mathbf{W}_{\text{enc}}\mathbf{f}_2 + \mathbf{b}_{\text{enc}}) = \begin{bmatrix} 0.7163 & -0.0997 & 0.5370 & -0.1974 \end{bmatrix}^T$$

Easy to get wrong

The real encoder is a bidirectional Mamba stack, and its arithmetic cannot be done on paper. Here it is replaced by the simplest function of the same *type* — DNA in, one vector out — so that the rest of the trace stays checkable. Everything after this point is the real mechanism, unchanged.

17 Stage 2 — Hi-C becomes contact strengths

The raw counts for a few pairs, the average at each separation, and the result of dividing one by the other:

Pair	Separation	Raw count	Expected	O/E	c_{ij}
(0, 1)	1	93.8	84.68	1.108	0.5255
(1, 2)	1	84.4	84.68	0.997	0.4992
(2, 3)	1	74.2	84.68	0.876	0.4670
(1, 3)	2	63.2	52.40	1.206	0.5467
(2, 4)	2	39.3	52.40	0.750	0.4286
(0, 5)	5	8.7	8.70	1.000	0.5000
(1,5)	4	66.4	40.90	1.623	0.6188

In plain words

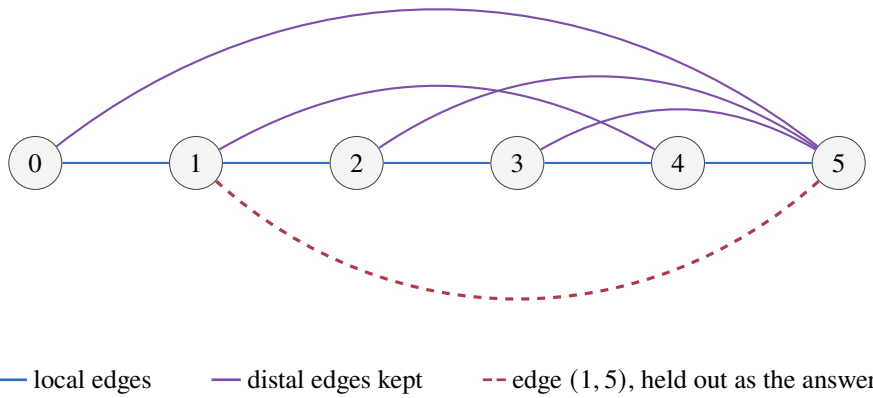
Look at the raw count for (1, 5) against (0, 1). In raw terms the neighbouring pair (0, 1) wins easily — of course it does, they are adjacent.

Now look at the O/E column. Pair (1, 5) has far more contact than is normal for things that far apart.

Detrending is what makes the loop visible. Before it, the loop is buried; after it, the loop is the largest number in the table.

18 Stage 3 — the graph

Local edges (radius 1), then each node's single strongest distal partner:



That leaves 9 edges in the conditioning graph. The window we are following, window 2, has neighbours **1, 3, 5**.

In plain words

Note what just happened. Window 2 can now talk to window 5, which is three bins away, because the Hi-C says they touch. A model reading the text alone would have no reason to connect them.

19 Stage 4 — contact-biased attention, in full

Window 2 now gathers information from its three neighbours. We compute each part of Equation (2) separately.

19.1 4a — the DNA part of the score

The query for window 2 is $\mathbf{q}_2 = [0.1476, -0.1886, 0.5329, -0.2815]$, and the scores against each neighbour's key are $\mathbf{q}_2^\top \mathbf{k}_j / \sqrt{4}$:

Neighbour j	DNA score	$\mathbf{b}_{\text{HiC}}$	$\mathbf{b}_{\text{dist}}$	total
1	0.0504	1.4975	-0.8000	0.7479
3	-0.0742	1.4011	-0.8000	0.5268
5	-0.1102	1.5119	-1.6000	-0.1982

In plain words

The DNA scores are all tiny — between -0.1102 and 0.0504 . At the start of training the encoder has learned nothing, so the sequence has almost nothing to say about who should talk to whom. The two bias terms are between ten and thirty times larger. **At the beginning of training, the structure is doing nearly all of the deciding.** As training proceeds and the DNA scores grow, the balance shifts. That is exactly the behaviour we want: start from what was measured, learn your way towards what can be read from sequence.

For this example the bias functions are kept linear so they can be differentiated by hand:

$$b_{\text{HiC}}(c) = 3.0 c, \quad b_{\text{dist}}(d) = -0.8 \log_2(d + 1).$$

19.2 4b — softmax

Take the largest total (0.7479) off every score first — this changes nothing mathematically, by Box 5, but stops e^x from overflowing on a computer. Then exponentiate, sum, divide:

Neighbour j	total – max	$e^{(\cdot)}$	weight a_{2j}
1	$0.7479 - 0.7479$	1.0000	0.4566
3	$0.5268 - 0.7479$	0.8016	0.3661
5	$-0.1982 - 0.7479$	0.3882	0.1773
sum = 2.1899			1.0000

In plain words

Window 2 takes **0.4566** of its incoming information from window 1, **0.3661** from window 3, and **0.1773** from window 5.

Window 5 has the *strongest* contact of the three ($b_{\text{HiC}} = 1.5119$) but is three bins away rather than one, so its distance penalty (-1.6000) pulls it back down. The two forces work against each other, exactly as designed.

Check it yourself

Take the three totals from the table in 4a. Subtract 0.7479 from each. Exponentiate. Add them up — you should get 2.1899 . Divide each by that total. You should recover 0.4566 , 0.3661 and 0.1773 , and they should add to 1.

19.3 4c — the weighted average

$$\text{output}_2 = 0.4566 \mathbf{v}_1 + 0.3661 \mathbf{v}_3 + 0.1773 \mathbf{v}_5 = \begin{bmatrix} 0.1545 & 0.1436 & -0.0077 & 0.0119 \end{bmatrix}^T$$

20 Stage 5 — residual connection and normalisation

We add the attention output back onto the original window vector, then normalise.

$$\mathbf{u} = \mathbf{h}_2 + \text{output}_2 = \begin{bmatrix} 0.8708 & 0.0439 & 0.5294 & -0.1855 \end{bmatrix}^T$$

In plain words

Why add the original back? So that information can flow through the layer untouched if that is the best thing to do. Without this, a layer that is not useful yet actively destroys what came before, and deep stacks fail to train.

It also matters here for a specific reason: when structure dropout removes the graph, this residual path is what keeps the block from collapsing.

Normalising means subtracting the mean and dividing by the standard deviation, so the four numbers have mean 0 and variance 1. For $\mathbf{u}$ the mean is 0.3146 and the variance is 0.1697:

$$\mathbf{u}_{\text{norm}} = \frac{\mathbf{u} - 0.3146}{\sqrt{0.1697 + \varepsilon}} = \begin{bmatrix} 1.3500 & -0.6573 & 0.5212 & -1.2139 \end{bmatrix}^T$$

with $\varepsilon = 10^{-5}$ present only so that we never divide by zero.

21 Stage 6 — feed-forward, and the final representation

The normalised vector goes through a small two-layer network — widen to 8 numbers, keep only the positives, project back to 4 — then another residual and normalisation. The result is the structure-informed representation of window 2:

$$\mathbf{z}_2 = \begin{bmatrix} 1.3559 & -0.6469 & 0.5092 & -1.2182 \end{bmatrix}^T$$

In plain words

This vector is the product of the whole project.

It began as 8 letters of DNA. It now also carries traces of the windows that physically touch this one — and it is a fixed-size list of numbers that any later task can use.

The research question is whether this vector is more useful than the one a sequence-only model would have produced. Not whether it looks different. Whether it is *more useful*, on tasks neither model was trained for.

22 Stage 7 — the three losses

22.1 Contact prediction on the held-out edge

The model never saw edge (1, 5). Using $\mathbf{z}_1$ and $\mathbf{z}_5$:

Head output (before squashing)	-0.6808
Predicted strength $\hat{c}$	0.3361
True strength c	0.6188
$\mathcal{L}_{\text{contact}}$	0.8309

The model predicted 0.3361 where the truth was 0.6188 — it badly under-estimated a real loop. That is expected: none of these weights has been trained. This number falling over the course of training is the model learning to read loops out of sequence.

22.2 Contrastive

The positive is the touching pair (1, 5), which spans 4 bins. The negatives are the other pairs spanning exactly 4 bins: (0,4).

Similarity of the positive pair	-0.7295
Similarity of the negative pair	0.6836
Temperature τ	0.5
$\mathcal{L}_{\text{contrast}}$	2.8838

Easy to get wrong

Distance matching compares pairs, not partners — and the first version of this example got it wrong.

The obvious reading of “distance-matched negatives” is: keep the same anchor, and pick other partners the same distance away. That fails here. From window 1 at a separation of 4 bins there is exactly one partner in a 6-window region, and it is the positive. The first draft silently fell back to a partner at a *different* separation — which is exactly the bias the matching exists to remove.

The correct comparison is pair against pair: hold the separation fixed, and ask whether *this* pair at 4 bins looks more like a contact than *other* pairs at 4 bins. docs/verify_math.py now asserts it.

22.3 Masked window

Window 3 is blanked and rebuilt from its neighbours: $\mathcal{L}_{\text{DNA}} = 0.2958$.

22.4 Total

The maths

$$\mathcal{L} = 1.0 \times 0.2958 + 0.5 \times 2.8838 + 1.0 \times 0.8309 = \boxed{2.5685}$$

23 Stage 8 — how the model learns to trust Hi-C

Training adjusts every number in the model to make $\mathcal{L}$ smaller, including α — the slope of b_{HiC} , which controls how much the contact map is allowed to influence attention at all.

The maths

Differentiating the softmax in Equation (2) with respect to α , where $b_{\text{HiC}}(c) = \alpha c$:

$$\frac{\partial a_{ij}}{\partial \alpha} = a_{ij} \left(c_{ij} - \sum_{m \in N(i)} a_{im} c_{im} \right) \quad (7)$$

Read it in words: *a neighbour whose contact strength is above the attention-weighted average gets more weight when α grows; one below it gets less.*

For window 2, the weighted average contact strength is 0.4883:

j	c_{2j}	$c_{2j} - 0.4883$	$\partial a_{2j} / \partial \alpha$
1	0.4992	$0.4992 - 0.4883$	0.0050
3	0.4670	$0.4670 - 0.4883$	-0.0078
5	0.5040	$0.5040 - 0.4883$	0.0028

These are not zero, so there is a real learning signal: the model can discover for itself how much the Hi-C is worth.

Easy to get wrong

This table was zero on the first run, and finding out why mattered.

The first version of the toy Hi-C was generated purely as a function of separation. After detrending, every O/E came out at exactly 1.0, so c_{ij} was identical for all three neighbours, so b_{HiC} was a constant — and by Box 5, softmax cancelled it. The gradient was exactly 0.

The lesson is not about toy data. **If the Hi-C carried no information beyond genomic distance, this mechanism would contribute literally nothing**, and the model would be mathematically unable to tell you so — it would simply train as though the term were absent.

That is precisely what the shuffled-Hi-C control in Section 24 is built to detect.

Part V — How we will know whether it worked

24 The controls

A number on its own means nothing. Every arm below is trained with the **same** parameter count, data, optimiser, schedule and step budget. Only the structural signal differs.

Arm	What changes	The question it answers
B0	No graph at all	Does structure help at all, at equal capacity?
B1	Edges placed at random	Does <i>any</i> graph help, regardless of meaning?
B2	Edges from separation only	Is the gain just distance — which the model can already read?
B3	Real contact values, shuffled	Does the actual biology matter? Same statistics, destroyed meaning.
B4	Hi-C added after encoding	Is conditioning inside the encoder better than gluing on afterwards?
B5	Hi-C only as an alignment target	Does our mechanism beat the closest published approach?
Ours	Contact-biased attention	—

In plain words

B3 is the one that decides the project. It receives exactly the same amount of extra input as our model, with the same distribution of values — only the pairing between values and edges has been scrambled.

If our model does not beat B3, then whatever helped was not chromatin biology, and we say so.

25 Hi-C-free inference — and a trap in the comparison

The model is run in three modes at the end:

Mode	DNA	Hi-C	What it tests
Sequence only	yes	no	Whether structure got into the DNA representation
Joint	yes	yes	Full multimodal capability
Structure only	no	yes	What the graph alone contributes

Easy to get wrong

The headline comparison must be made with Hi-C switched off for both sides.

It is tempting to report our model *with* its graph against the sequence-only baseline B0. But then our model has been handed part of the answer at test time and B0 has not. It would win, the win would prove nothing, and a reviewer would say so in one sentence.

The fair and meaningful comparison is: both models run on sequence alone, with the only difference being what shaped their weights during pretraining. Numbers measured with the graph available are still worth reporting — as an upper bound, labelled as such.

26 What would falsify the claim

Stated in advance, so it can lose:

- Our model failing to beat **B3** (shuffled Hi-C) on long-range contact prediction.
- Our model merely matching **B2** (distance only) — meaning it learned how far apart windows are, not how they fold.
- No advantage surviving into sequence-only inference — meaning we built a multimodal predictor, not a better DNA representation.

In plain words

Any of these outcomes gets reported as measured. A clean negative result on a well-controlled experiment is a real finding about the limits of the signal. A positive result from an uncontrolled experiment is not a finding at all.

27 Honest positioning

Contact-guided attention is **not** new. Chromoformer used chromatin interactions to guide attention for regulatory prediction; GraphReg propagated genomic features over Hi-C-derived graphs; CHROME uses filtered contact graphs with graph attention. Evo2HiC, the closest precedent, uses Hi-C to guide contrastive distillation from a large DNA model into a compact encoder.

Easy to get wrong

None of the following may be claimed:

- the first genomic foundation model to use Hi-C;

- the first Hi-C foundation model;
- the first model to combine DNA sequence and Hi-C;
- the invention of contact-guided attention.

Each is contradicted by published work.

What we do claim is narrower and testable:

A compact, end-to-end, self-supervised model combining a bidirectional Mamba DNA encoder with sparse Hi-C-conditioned information flow — evaluated for whether the resulting DNA representation transfers across genomic tasks and retains structural information when Hi-C is unavailable at inference.

The contribution is the *integration and its evaluation*, not the individual components.

Appendix

A Notation

Symbol	Meaning
i, j	indices of genomic windows
$\mathbf{h}_i$	the vector the local encoder produces for window i
$\mathbf{z}_i$	the structure-informed representation, after the encoder block
$\mathbf{q}_i, \mathbf{k}_j, \mathbf{v}_j$	query, key and value vectors
d	the dimension of those vectors (4 here, 256 in the real model)
c_{ij}	detrended contact strength between windows i and j , in $(0, 1)$
d_{ij}	their separation, counted in bins
a_{ij}	attention weight: how much i takes from j
$\mathcal{N}(i)$	the graph neighbours of window i
$b_{\text{HiC}}, b_{\text{dist}}$	the learned bias functions
α	the slope of b_{HiC} in the worked example
τ	contrastive temperature
$\lambda_\bullet$	the weight of each loss term
$\sigma(x)$	the logistic function $1/(1 + e^{-x})$

B Glossary

Base / nucleotide

One letter of DNA: A, C, G or T.

Bin

A fixed-size chunk of the genome used by Hi-C. Ours are 5 kb.

Chromatin

DNA together with the proteins it is wrapped around — DNA as it actually exists in a cell, rather than as an abstract sequence.

Contact map

The table of Hi-C counts: how often each pair of bins was found touching.

CTCF

A protein that binds specific DNA motifs and helps anchor chromatin loops. Its motif appearing at both ends of a loop is a strong signature.

Detrending

Dividing observed contact counts by the average expected at that separation, to remove the background distance effect.

Embedding

A learned vector summary of something.

Enhancer

A DNA region that increases the activity of a gene, often from far away.

Foundation model

A model pretrained once on broad data, then reused for many tasks.

hg38

The human reference genome assembly this project uses.

Hi-C

The experiment that measures which genomic regions are physically close.

Loop

A chromatin structure bringing two distant genomic regions together.

Mamba

A state space sequence model; costs time proportional to length rather than length squared.

Motif

A short recurring DNA pattern that a protein recognises.

O/E

Observed over expected — a contact count divided by the average at its separation.

Promoter

The region just in front of a gene where transcription starts.

Self-supervised learning

Training by hiding part of the input and predicting it, requiring no human labels.

Softmax

A function turning a list of scores into positive weights that sum to one.

Window

One 5 kb piece of DNA; also one node of our graph.

C Reproducing the worked example

File	What it holds
docs/worked_example.py	the whole toy forward pass
docs/generated/we_values.tex	every number in Part IV, as $\text{\LaTeX}$ macros
docs/generated/we_values.json	the same values, machine readable
docs/verify_math.py	32 numerical checks on the mathematics above
configs/base.yaml	the real model's settings

Run `python docs/worked_example.py`, then compile this document twice. Not one number in Part IV is typed into the $\text{\LaTeX}$ source, so the write-up cannot disagree with the arithmetic that produced it.

`python docs/verify_math.py` re-derives the claims made above and checks them numerically: that the softmax weights sum to one and are shift-invariant, that a constant b_{scale} term provably cancels, that Equation (7) agrees with a central finite difference, that every LayerNorm output has mean 0 and variance 1, that observed-over-expected averages to 1 at every separation, that the held-out edge is absent from the conditioning graph, that the contrastive negatives really are distance-matched, and that the losses match their stated formulae. All 32 checks pass.

*Every result table in this project stays marked ??
until a file under `results/` produces the number.*